

Dashboard compartilhado, sem gasto

Plano completo para concluir a integração do MacBreak entre aparelhos, mantendo o projeto Firebase mequi-break no plano Spark. Estado verificado no repositório em 03/10/2026.

Objetivo de entrega

Cada partida dos três minigames gera uma sessão local identificada pelo ID da instalação. Quando houver internet, essa sessão entra no Firestore uma única vez. O dashboard principal mostra, por unidade, mês e turno, os totais somados de todos os aparelhos. A opção Demonstração continua separada.

Situação real agora

Já preparado no projeto	Ainda falta para funcionar entre aparelhos
Firebase Unity SDK 13.17.0, arquivo de configuração Android e ID local por instalação.	Confirmar Spark e ativar o provedor de Authentication anônima no console.
Registro local das sessões; sincronizador com reenvio e transação que evita duplicação.	Examinar as regras atuais do banco e publicar as regras testadas para o projeto certo.
Dashboard2 ligado ao hub; leitura dos totais em tempo real e modo Demonstração preservado.	Gerar build Android e validar em dois aparelhos reais, com conexão e sem conexão.
Regras do Firestore e testes no emulador; compilação Unity em lote bem-sucedida.	Medir leituras, escritas, tamanho da build e comportamento nas cotas gratuitas.

Fluxo previsto

1. Joga → 2. Salva localmente → 3. Sincroniza → 4. Exibe

Nome, unidade e turno continuam no login atual. Um ID persistente representa a instalação; um usuário anônimo do Firebase autoriza o acesso técnico ao banco. A transação grava a sessão privada e soma seus contadores nos documentos de todos os turnos e do turno escolhido. O dashboard escuta somente os totais.

Limite de escopo: nenhuma cobrança, cartão ou upgrade para Blaze; nenhum Cloud Functions. Se uma cota Spark for atingida, a sincronização poderá parar até a cota se renovar. A sessão local deve permanecer guardada.

01 Preparação e decisões

Antes de publicar qualquer regra ou distribuir uma build.

Passo 1 - verificar o projeto no console Firebase

- Confirmar que o projeto **mequi-break** segue em **Spark**, sem conta de faturamento vinculada. Registrar uma captura da tela do plano.
- Confirmar o aplicativo Android cadastrado com identificador **com.DefaultCompany.MQUIBREAK** e conferir se o `google-services.json` do projeto pertence a esse cadastro.
- Confirmar que o Cloud Firestore já existe no banco padrão; anotar a região configurada. Não criar outro banco para esta demonstração.
- Em **Authentication** → **Sign-in method**, ativar **Anônimo**. Isso ocorre nos bastidores: o funcionário não precisa de e-mail nem senha.
- Ler e guardar uma cópia das regras **publicadas hoje** antes de substituí-las. Verificar se há outras coleções no mesmo banco que dependam delas.

Passo 2 - fechar a identificação das unidades

Hoje a unidade é texto livre. O código remove espaços nas pontas, converte para maiúsculas e calcula uma chave. Assim, “Loja 101” e “LOJA 101” se unem, mas “Loja-101” e “Loja 101” se separam. Antes do teste, definir **um código único por unidade** e usá-lo igual em todos os celulares. Depois, substituir texto livre por seleção/validação simples, se necessário.

Passo 3 - decidir o tratamento do histórico

Sessões antigas sem ID da instalação

Elas permanecem no histórico local e não são enviadas automaticamente. Atribuí-las ao aparelho atual falsearia a origem. O teste compartilhado começa com sessões novas, criadas pela versão com ID. Qualquer migração futura exige uma regra explícita e revisão dos dados.

Critério para avançar

Projeto e aplicativo corretos confirmados; Spark confirmado; Authentication anônima ligada; regra publicada atual salva; código de unidade definido para os aparelhos de teste.

02 Publicação e build

Usar somente Firestore e Authentication no plano gratuito.

Passo 4 - publicar as regras

- Comparar o arquivo **firestore.rules** do repositório com as regras em produção. Manter qualquer acesso legítimo já existente; não publicar cegamente se o banco for compartilhado por outra função.
- Publicar apenas regras do Firestore, pelo console ou CLI autenticada pelo dono do projeto. O arquivo **firebase.json** aponta somente para essas regras; não há deploy de Functions.
- As regras previstas permitem criar uma sessão privada com dois totais correspondentes numa operação atômica, bloqueiam atualização e exclusão posteriores e deixam cada sessão legível apenas para seu usuário anônimo. Os totais podem ser lidos por aparelhos autenticados na demonstração.
- Confirmar no console que as regras ativas têm a versão esperada e repetir os cenários do emulador contra a configuração de teste antes de distribuir a build.

Passo 5 - fechar a build Android

- Abrir o projeto na versão de Unity configurada, resolver as dependências do Firebase e verificar se App, Auth e Firestore entram no Android sem conflito.
- Gerar APK de teste, instalar em um aparelho limpo e conferir inicialização, login atual, três minigames, tela de resultado, retorno ao hub e Dashboard2.
- Conferir que o dashboard mostra **Neste aparelho** quando os totais da nuvem ainda não chegaram e **Minha loja** quando a leitura compartilhada está pronta. A Demonstração deve continuar usando dados fictícios separados.
- Comparar tamanho do APK gerado com a build anterior. Registrar o aumento real causado pelo SDK em vez de estimá-lo.

Resultado esperado da fase

Marco técnico

Uma build Android abre sem erro, autentica silenciosamente no Firebase, registra uma nova sessão após jogar e cria exatamente três documentos por sessão: um documento individual e dois agregados (todos os turnos e o turno da partida). O dashboard lê os agregados.

03 Teste de ponta a ponta

Executar com dois celulares e uma unidade de teste padronizada.

Cenário	Procedimento	Aprovação
Aparelhos A e B	Entrar com a mesma unidade; jogar sessões nos três minigames em turnos diferentes.	Dashboard de ambos mostra a soma; filtros de manhã/tarde/noite e todos batem.
Unidades separadas	Mudar apenas a unidade no aparelho B e criar nova sessão.	Os dados de B não entram nos totais da unidade de A.
Sem internet	Jogar offline, fechar/reabrir o app, reconectar.	Sessão local preservada; upload posterior sem perda nem duplicação.
Reenvio e falha	Interromper a rede durante a sincronização; repetir abertura/sincronização.	Mesmo ID de sessão é contado uma vez. Erro transitório não apaga dados locais.
Privacidade	Usar uma instalação para tentar ler a sessão individual de outra.	Leitura individual negada; somente totais ficam visíveis no dashboard.
Período e visual	Ver mês atual/anterior, todos os turnos e cada turno; alternar Demonstração.	Gráficos, barras, percentuais e comparações refletem os dados escolhidos.

Conferência das porcentagens

Somar **contadores** antes de calcular cada percentual. Exemplo: A atende 9 de 10 pedidos; B atende 1 de 2. A loja atende 10 de 12 = **83,3%**. A média simples de 90% e 50% (70%) seria errada. Repetir esse cálculo nos cartões de Rush Balance, Combo Crew e nas escolhas do Energy Station.

Evidências a guardar

- Prints do console com as três gravações de uma sessão e com os totais após partidas de A e B, sem expor nome do funcionário.
- Vídeo curto da atualização do Dashboard2 em ambos os aparelhos e dos filtros de período/turno.
- Registro de uma partida offline que aparece depois da reconexão, com o mesmo ID e sem contagem dupla.
- Resultado da comparação numérica entre sessões locais e agregados, por jogo, unidade, mês e turno.

04 Operação no Spark

Controlar uso, qualidade dos dados e continuidade sem faturamento.

Cotas atuais a acompanhar no console

Recurso Firestore gratuito	Limite de referência
Dados armazenados	1 GiB
Leituras de documentos	50.000 por dia
Escritas de documentos	20.000 por dia
Exclusões de documentos	20.000 por dia
Transferência de saída	10 GiB por mês

Uma sessão nova escreve três documentos. A transação e as regras também leem documentos. O Dashboard2 hoje abre **36 listeners** (3 meses × 4 seleções de turno × 3 jogos) por aparelho com o dashboard aberto. Medir uso real antes de otimizar; se necessário, ouvir apenas o período e a aba visíveis. Não ativar TTL, backups, PITR ou serviços que exijam faturamento.

Rotina de verificação após liberar a demonstração

- Acompanhar diariamente as métricas de leituras, escritas e armazenamento durante o piloto; registrar número de aparelhos e partidas.
- Checar amostras semanais de sessões pendentes no celular versus totais do Firestore. Corrigir falhas de conexão sem descartar o registro local.
- Padronizar a unidade no login. Revisar relógio/fuso do aparelho: o mês é calculado a partir da data da sessão convertida para horário local no envio.
- Confirmar que o projeto permanece Spark. Ao aproximar-se da cota, reduzir leituras e limitar a escala do piloto; não migrar automaticamente para Blaze.

Definição de concluído

Pronto para testar com a equipe

Duas instalações distintas somam sessões reais no mesmo dashboard; os três minigames e todos os filtros têm números corretos; offline recupera; duplicatas não contam duas vezes; regras protegem sessões individuais; a Demonstração segue intacta; projeto continua Spark e a build Android não regrediu.

05 Pendências, riscos e responsáveis

Lista de execução em ordem, com condições de parada claras.

Ordem	Tarefa / responsável	Dependência / saída
1	Proprietário: confirmar Spark, aplicativo e Firestore; ativar Authentication anônima.	Acesso ao console. Não fornecer senha ou códigos no chat.
2	Implementação: guardar regras atuais, comparar e publicar as regras do repositório.	Parar se outras coleções forem afetadas; ajustar antes do deploy.
3	Implementação: gerar APK, verificar SDK e fazer teste em um aparelho.	Se Auth/regras falharem, preservar dados locais e corrigir.
4	Equipe + implementação: testar dois aparelhos, três jogos, turnos, unidades e offline.	Aprovar cálculos e reconciliação sessão → total.
5	Implementação: medir tamanho do APK e consumo Spark; ajustar listeners se necessário.	Relatório de uso do piloto e confirmação de gasto zero.

Riscos conhecidos e tratamento

- **Unidade digitada de formas diferentes:** divide os resultados. Usar código único e depois restringir a entrada.
- **Transação offline:** não conclui naquele momento. Manter a sessão local e reenviar ao voltar a conexão.
- **Cliente alterado intencionalmente:** as regras restringem formato e soma, mas não provam que a partida foi jogada. Aceitável somente para a demonstração aberta; reavaliar para produção.
- **Alta concorrência nos mesmos totais:** transações podem ser repetidas ou falhar; observar no piloto e reestruturar apenas se houver evidência.
- **Histórico sem ID:** fica local. Não atribuir retroativamente um aparelho.
- **Cota gratuita excedida:** serviço pode ser interrompido até a renovação; dados locais permanecem pendentes.

Referências e arquivos verificados

Projeto: **Packages/manifest.json**; **Assets/Scripts/Data/FirebaseSessionSync.cs**; **Assets/Scripts/Dashboard/DashboardCloudData.cs**; **Assets/Scripts/Dashboard/DashboardExperimentController.cs**; **firestore.rules**; **firebase.json**.

Documentação oficial: firebase.google.com/docs/projects/billing/firebase-pricing-plans • firebase.google.com/docs/firestore/quotas • firebase.google.com/docs/auth/unity/anonymous-auth • firebase.google.com/docs/firestore/solutions/aggregation

Fora do escopo agora: contas de gerência, e-mail/senha para funcionários, Cloud Functions, migração automática do histórico antigo e mudança para plano pago. Essas decisões podem ser revisitadas após o piloto, com dados reais.